AI Engineering and Claude Architect · Part I, Foundations

Generative AI and LLMs, Lesson 5

Inference and Model Serving Internals

Your first token takes 900 milliseconds, then the rest arrive smoothly. Long replies cost more than long questions. And a batch job costs half price. Each of these is a direct consequence of how a GPU generates text, one token at a time.

Module	AI02 of 13
Lesson	5 of 7
Prerequisites	AI02 L2–L3. HLD M21D recommended
Exam relevance	Serving internals and streaming implementation are out of scope. `stop_reason` and batch trade-offs are tested (TS 1.1, TS 4.5)
Reading time	~40 minutes

Contents

1. Inference and Model Serving Internals

Lesson 5 of 7 · Generative AI and LLM Fundamentals

Inference and Model Serving Internals

Key idea

An LLM generates one token at a time: run the model, get a probability for every token, choose one, append it, repeat. Because the choice is sampled, output varies. Because everything before must be available at each step, a **KV cache** avoids recomputing it, at a large memory cost. And since requests vary wildly in length, **continuous batching keeps the GPU busy by swapping requests in and out at each step.** These mechanics explain the latency, price and API features you see.

> *Before: the transformer (AI02 L3). HLD M21D covers KV cache sizing, GPU memory and fleet scaling in depth, and this lesson doesn't repeat them. Teaches: the generation loop, sampling and decoding, stop reasons, the KV cache's saving, batching, serving metrics and what they mean when you call an API. Used by: AI03 L1, AI03 L5, AI08 L2, AI10 and AI11 L2.*

The hook: three mysteries

Your team has just started calling Claude, and has three questions:

1. "Why is the same request sometimes answered differently?"
2. "Why does the reply stream word by word, rather than arriving complete?"
3. "Why is the Message Batches API half price, if it's the same model?"

All three have the same root: generation is a loop on expensive shared hardware.

Intuition: an autocomplete with a very expensive keyboard

Think of your phone's next-word suggestion. Tap a suggestion, and it offers new ones. An LLM does just that, with two differences. Its suggestions are astonishingly good. And each tap costs a run through a huge network on a GPU that many other people are also waiting for.

The generation loop

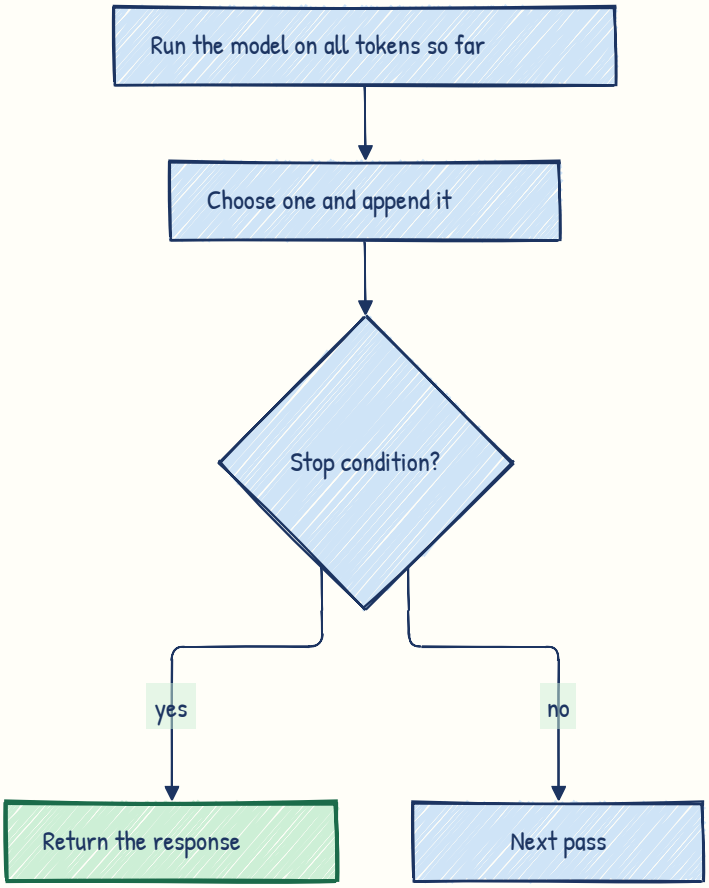


Figure 1. The generation loop. Each pass produces exactly one token. It repeats until a stop condition is met.

Two consequences follow directly:

→ **Streaming is natural.** Tokens exist one at a time, so sending them as they arrive is simply exposing the loop.

→ **Output is typically slower than input.** A prompt can be processed in parallel, while output tokens are produced one after another. That helps explain why output tokens usually cost more than input

tokens. For Claude Opus 5.5, the listed prices are $4 and $20 per million tokens, as of the October 2026 reference.

Choosing the next token

The model outputs scores, softmax makes them probabilities, and a **decoding strategy** chooses. Real output from `code/ai02-llm/Serving.java`, showing the probabilities for four candidates:

```
tokens: [approved, denied, pending, banana]
T=1.0         0.591 0.266 0.132 0.011
T=0.3         0.929 0.065 0.006 0.000
T=2.0         0.439 0.294 0.207 0.059
top-k k=2     0.690 0.310 0.000 0.000
top-p p=0.9   0.598 0.269 0.133 0.000
```

Strategy	How it works	What the output above shows	Effect
Greedy	Always take the top token	Always `approved`	Predictable, can be dull or repetitive
Temperature	Divide scores by T before softmax	Low T sharpens, high T flattens	Controls randomness
Top-k	Keep only the k best, then renormalize	`pending` and `banana` removed	Stops very unlikely tokens
Top-p (nucleus)	Keep the smallest set reaching probability p	`banana` removed	Adapts to how sure the model is
Beam search	Track several candidate sequences	Not shown	Common in translation. Less used for chat

Now you can answer the first mystery. **The same input can differ because the next token is sampled.** At T=1.0, `denied` has a 26.6% chance, even though `approved` is the favorite.

What's different on current Claude models

You won't set temperature on current Claude models. Sampling parameters are rejected with a 400 on API version `2026-10-21`. The control you get instead is `effort`, inside `output_config` (AI03 L4). Other providers and open models still expose these settings, so **know the concept, but check the API.**

When generation stops

Your code **must** check why generation stopped. These are the values to handle on the Claude API, and the exam tests the first two:

stop_reason	Meaning	What your code does
`end_turn`	The model finished	Use the response
`tool_use`	The model wants a tool run	Execute it, and continue the loop (AI06 L2)
`max_tokens`	Hit your limit	Output is truncated, so don't treat it as complete
`stop_sequence`	Hit a stop string you set	Use the response, which ends there
`pause_turn`	A long server-tool turn paused	Send it back to continue
`refusal`	A safety classifier declined	Check this before reading content, then fall back

The classic bug

A team parses JSON from every reply. It works until a long reply stops at `max_tokens`, half a JSON object arrives, and the parser fails in production. Always check `stop_reason` first.

The KV cache: why generating isn't absurdly slow

At each step, the new token needs keys and values from all earlier tokens (AI02 L2). Recomputing them every step would be wasteful, since they don't change. So servers save them, in the **KV cache**. Real output, counting tokens run through the model:

```
prompt=2000, output=500: processed without cache=1,124,750, with=2,500 (449x)
prompt=100000, output=500: processed without cache=50,124,750, with=100,500 (498x)
```

The cache makes it hundreds of times cheaper, in this measure. The cost moves to memory: HLD M21D shows that a large model can need about 38 GB of cache for a single 128K-token chat, and that GPU memory rather than compute often caps the batch size. Read M21D's "the KV cache" section for that calculation.

In the real world

The vLLM paper (Kwon and colleagues, 2023) describes the KV cache as memory that's "huge and grows and shrinks dynamically", with fragmentation limiting batch size. Its answer, PagedAttention, borrows virtual memory and paging from operating systems. It reports 2–4 times higher throughput than the compared systems at similar latency. For a backend engineer, that's reassuring: an OS idea solved an AI bottleneck.

This connects to prompt caching. If many requests share a long prefix, the server can reuse that prefix's work. That's the principle behind prompt caching on the API (AI03 L5). The API's implementation isn't public in detail, so treat this as the intuition, not a description of Anthropic's internals.

Batching: keeping an expensive GPU busy

GPUs are efficient when they process many sequences together. The difficulty is that requests vary in length, and that matters a lot. This simulation gives 8 slots to 200 requests, with 80% short and 20% long, all waiting from the start. Real output:

```
static     steps=13,106 mean completion=   6,661 slot utilization=30%
continuous steps= 4,347 mean completion=   1,910 slot utilization=90%
```

Method	How it works	Result in this simulation
Static batching	A batch runs until its longest request ends	Finished slots sit idle, at 30% utilization
Continuous batching	As soon as one request ends, a new one takes its slot	90% utilization, with about a third of the steps

```
while (!waiting.isEmpty() || !active.isEmpty()) {
    while (active.size() < slots && !waiting.isEmpty()) {
        active.add(new int[] {waiting.poll()});
    }
    clock++;
    busy += active.size();
    for (int[] slot : active) {
        slot[0]--;
    }
    for (int i = active.size() - 1; i >= 0; i--) {
        if (active.get(i)[0] == 0) {
            active.remove(i);
            latency += clock;
        }
    }
}
```

These are simulated steps with invented request lengths, so don't quote the percentages as real serving figures. **The principle is general: the shape of the workload determines the waste.**

Other serving optimizations

Technique	Idea	Trade-off
Quantization	Store weights at lower precision	Smaller and faster, with possible quality loss (AI11 L2)
Speculative decoding	A small model drafts tokens, and the large model verifies them in bulk	Faster when drafts are good
Prefix caching	Reuse computed prefixes	Needs routing requests back to the right cache
Tensor parallelism	Split one model across GPUs	Needed when weights don't fit, per HLD M21D

The metrics that describe serving

Metric	Meaning	What drives it
TTFT (time to first token)	How long until output begins	Queueing, plus input length
Inter-token latency	Gap between tokens	Batch load and model size
Total latency	Full response time	TTFT + output tokens × inter-token gap
Throughput	Tokens per second per GPU	Batching and optimizations

A typical formula for planning:

```
total ≈ TTFT + output tokens / output speed
```

For a chat UI, optimize TTFT, and stream. For a pipeline, optimize total latency and cost.

How Anthropic approaches it

Anthropic doesn't publish how its serving fleet is built. It does expose controls that reflect these mechanics, and each answers a real problem.

Physical fact	API feature	What you get
GPU capacity is scarce at peak	Message Batches: flexible timing, up to 24 hours	50% lower cost, and no latency guarantee (AI03 L5)
Long prefixes repeat	Prompt caching	Reduced cost on repeated prefixes (AI03 L5)
Generation is sequential	Streaming	Fast perceived response
Very long outputs risk HTTP timeouts	SDKs require streaming for very large `max_tokens`	Connection stays active
Speed can be traded for price	Fast mode, a research preview on some Opus models	Up to 2.5× output speed at premium pricing
Depth of reasoning varies by task	`effort`	Trade thoroughness against spend (AI03 L4)

So the third mystery has an answer, as a reasonable inference: batch work tolerates waiting, so it can run when capacity allows, and that flexibility is worth a discount. Anthropic documents the discount and the window, not its internal scheduling.

The production architecture

HLD M21D gives the complete design for **operating** an LLM fleet, including routing and GPU scaling. Here's the other view: **you're the client of a hosted model.** **This is a recommended reference design, not a specific company's system.**

Baseline: a blocking call from the request thread. Simple, but a long generation holds a thread, users stare at a spinner, and an overload propagates into your service.

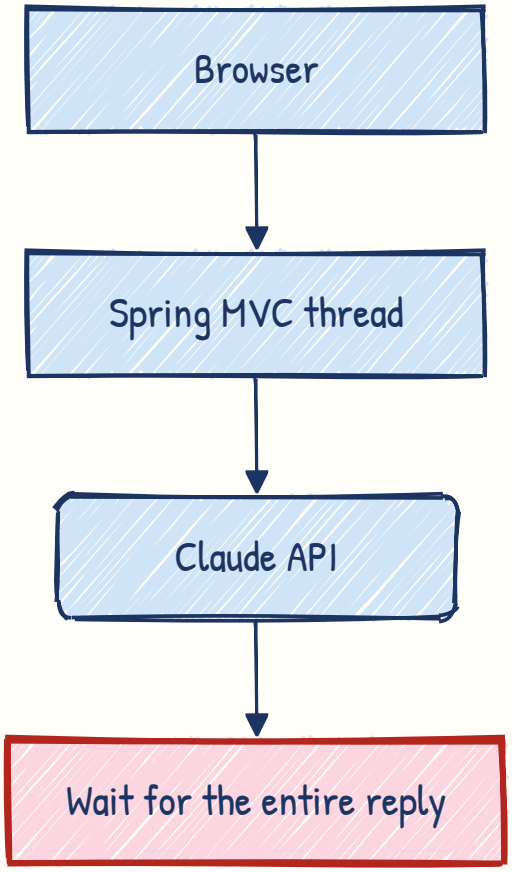


Figure 2. Baseline. A request thread blocks for the whole generation, with no streaming and no protection from slow responses.

Improved: streaming to the browser. TTFT is now what users feel. New problem: **all work is treated alike, so overnight jobs compete with live chat for the same rate limit, at full price.**

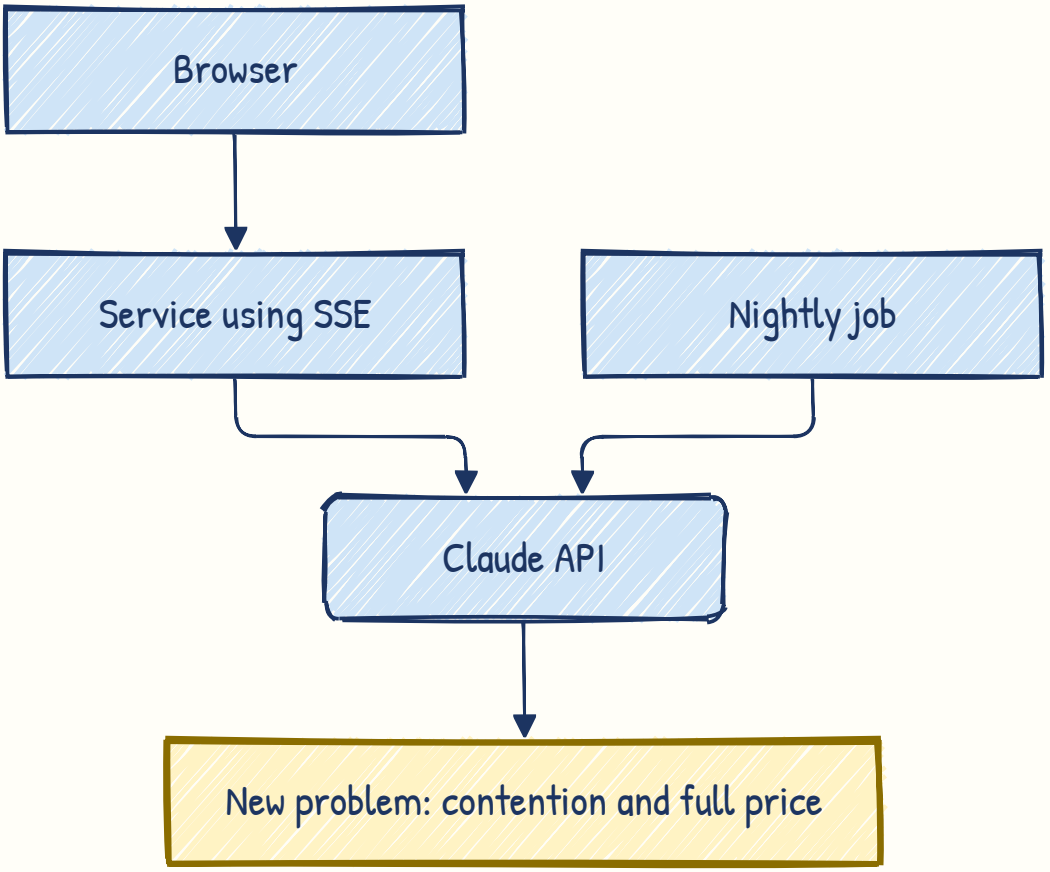


Figure 3. Improved. Streaming fixes perceived latency, but interactive and offline work now share one limit.

Production: route by latency need.

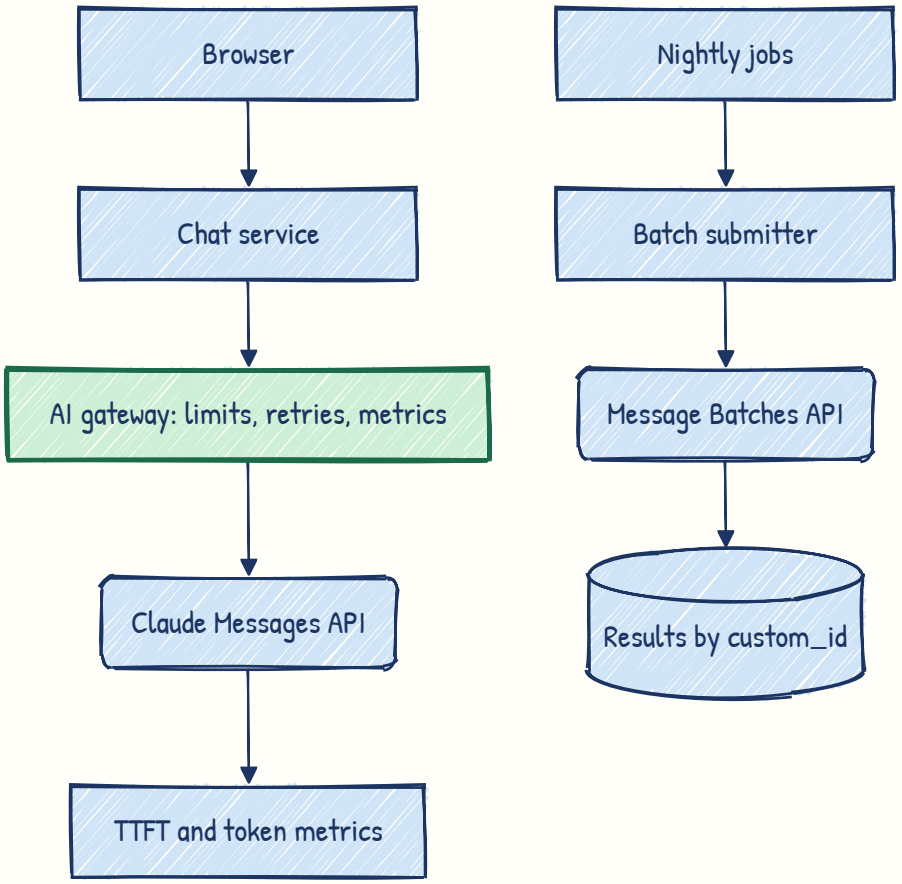


Figure 4. Production. Live traffic streams through a gateway. Offline work goes to the Batches API at half price, and each path has its own limits.

Component	Why it exists	How it communicates
AI gateway	One place for retries, limits and metrics (AI10 L2)	HTTP, streaming through
Chat service	Holds the conversation and streams to users	SSE to the browser (AI08 L2)
Batch submitter	Sends latency-tolerant work at 50% lower cost	Submits batches, polls for completion
Results store	Batch results can arrive in any order	Key by `custom_id`, never by position
Metrics	TTFT, tokens and `stop_reason` show what's really happening	Async, into your metrics stack

Failures

Failure	Effect	Containment
`max_tokens` reached	Truncated output	Check `stop_reason`, then raise the limit or continue
Stream breaks midway	Partial reply	Keep the partial text, and let the user retry (AI08 L2)
HTTP 429 or overload	Rejected or delayed	The SDK retries, then back off, then fall back (AI10 L1)
`refusal`	No usable content	Check first, then handle it as a business outcome
Batch misses your SLA	Late results	Submit early enough: the window can be up to 24 hours (AI03 L5)

Java

Check why generation stopped before using the output. Sample pattern, using the SDK types from `code/ai05-rag`, which compile against `anthropic-java` 3.2.0:

```
Message reply = client.messages().create(params);
if (!reply.stopReason().filter(StopReason.END_TURN::equals).isPresent()) {
    return Optional.empty();
}
```

A real service would branch on each reason, and AI08 L1 builds that. Streaming and virtual threads are in AI08 L2.

Trade-offs

Decision	Gain	Cost
Streaming	Better perceived speed	More client code, and mid-stream failures
Batch API	50% savings	Up to 24 hours, no multi-turn tools
Larger `max_tokens`	Avoids truncation	Higher maximum cost for a runaway reply
Self-hosted serving	Control over data and cost at scale	You now own everything in HLD M21D

When not to self-host: unless you have sustained, high volume, or a hard data residency constraint, a hosted API is usually cheaper once you count the operational work (AI11 L2). That's an engineering recommendation, and worth testing with your own numbers.

Common mistakes

1. **Reading content before checking `stop_reason`.** Truncated or refused output will break you later.
2. **Treating output as deterministic.** Test statistically (AI10 L3).
3. **Sending temperature to current Claude models.** You'll get a 400. Use `effort`.
4. **Using a batch for a blocking workflow.** There's no latency guarantee.
5. **Matching batch results by position.** Results can arrive in any order, so use `custom_id`.
6. **Measuring only total latency.** Users feel TTFT.

What to say in an interview

For "why stream?": "Generation is sequential, so total time depends on output length, but users feel TTFT. Streaming surfaces tokens as they're produced." For "why is output pricier than input?": "Output tokens are produced one at a time, while input can be processed in parallel. It's a plausible reason, though providers don't publish their pricing logic." Saying what's inference earns trust.

Certification lens

How this lesson maps to CCAR-F

Tested: `stop_reason` drives control flow (TS 1.1), and batch is only for latency-tolerant work: 50% savings, up to 24 hours, no latency SLA, no multi-turn tools, correlated by `custom_id` (TS 4.5). **Out of scope:** streaming implementation, and serving internals.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Claude Code for Continuous Integration · Medium. Two workflows run on Claude: a blocking pre-commit check, and a weekly security audit report. Finance wants 50% savings on both via the Message Batches API. What should you recommend?

A. Move both to batch, with polling every minute.

B. Move only the weekly audit to batch. Keep the synchronous API for the blocking check.

C. Keep both synchronous, since batch can't correlate results with requests.

D. Move both to batch, and fall back to the synchronous API after a timeout.

A **B.** Match the API to each workflow's latency needs (TS 4.5). Batch has no latency guarantee, so A blocks developers. C is wrong because `custom_id` correlates results. D adds complexity, when the simple split solves it.

Q Q2 · Code analysis · Medium. A pipeline calls `mapper.readValue(firstTextBlock, Invoice.class)` immediately on every response. It fails rarely, on long invoices, with "Unexpected end of input". What's the cause, and the fix?

A The reply reached `max_tokens`, so the JSON is cut off. Check `stop_reason` before parsing, and raise the limit or split the document. Using tool use or structured outputs (AI03 L3) also helps with syntax, but truncation still has to be handled.

Q Q3 · Calculation · Medium. TTFT is 800 ms, and output runs at 50 tokens per second. Approximately how long does a 500-token reply take? What will the user notice first?

A 0.8 + 500/50 = **10.8 seconds**. With streaming, the user sees the first words after 0.8 seconds. Without it, they wait the full 10.8.

Revision - Lesson 5

→ Generation loop: run the model, get probabilities, choose a token, append, repeat.

→ Sampling explains why the same input can produce different output.

→ Temperature sharpens or flattens. Top-k and top-p remove unlikely tokens.

→ Current Claude models reject sampling parameters. `effort` is the control.

→ Stop reasons: end_turn, tool_use, max_tokens, stop_sequence, pause_turn, refusal. Check them first.

→ The KV cache avoids recomputation, hundreds of times over in the demo, at a large memory cost.

→ Continuous batching fills freed slots at once: 90% utilization versus 30% in the simulation.

→ TTFT is what users feel, and total ≈ TTFT + output tokens / speed.

→ Batch: 50% cheaper, up to 24 hours, no latency SLA, correlate by custom_id.

→ Route by latency need: live traffic streams, and offline work goes to batch.

Your notes